

Chapitre 3

Processus & Threads

Système d'Exploitation et Programmation Système

Pr. MANALE Boughanja
m.boughanja@ump.ac.ma

Table des matières

1 Terminologie	2
2 Définition d'un Processus	2
2.1 Exemple : Processus de Comptage	2
3 Chargement du Programme en Mémoire	2
3.1 Mémoire Virtuelle	3
3.2 Processus de Chargement	3
3.3 Exemple de Passage Adresse Linéaire → Adresse Virtuelle	3
4 Gestion des Processus par le Système	3
4.1 Commande ps	3
5 États d'un Processus	4
6 Ordonnancement des Processus	4
6.1 Les Niveaux d'Ordonnancement	4
6.2 Types d'Ordonnancement	4
7 Communication et Synchronisation des Processus	5
7.1 Problème : Condition de Course (Race Condition)	5
7.2 Techniques de Synchronisation	5
7.3 Le Mutex (Exclusion Mutuelle)	5
7.4 Les Sémaphores	6
8 Programme vs Processus	6
8.1 Création de Processus	6
9 Exécution d'un Processus	6
9.1 Exécution en Série (Séquentielle)	6
9.2 Exécution en Parallèle (Concurrente)	7
9.3 Exécution Pipe	7
10 Algorithmes d'Ordonnancement	7
10.1 FCFS (First-Come First-Served)	7
10.2 Shortest Job First (SJF)	8
10.3 Shortest Remaining Time (SRT)	8
10.4 Round Robin	8
10.4.1 Exemple : Round Robin	8
10.5 L'Ordonnancement avec Priorité	9
10.5.1 Exemple : Ordonnancement avec Priorité	9

1 Terminologie

Définition

Programme : Un fichier source écrit dans un langage donné (C, C++, Java, etc.) ou le fichier en langage binaire obtenu après compilation (programme exécutable).

Définition

Processus : Un programme en cours d'exécution par le système d'exploitation.

Définition

Thread : L'unité d'exécution la plus petite à l'intérieur d'un processus. Un processus peut être décomposé en plusieurs threads.

2 Définition d'un Processus

Définition

Un processus est constitué d'un **programme binaire** (instructions + données statiques), associé à un **contexte d'exécution** :

- Pile + Tas
- Données dans les registres du processeur
- Instruction courante

2.1 Exemple : Processus de Comptage

Exemple

Scénario : Un programme qui compte de 1 à 5.

Composant	Description
Programme Binaire	Instructions : Compter 1, puis 2, puis 3, etc. Données Statiques : Le nombre maximum est 5.
Pile + Tas	Stoque temporairement les nombres qui doivent être affichés.
Registres du Processeur	Contient le nombre actuel en traitement (ex : le nombre 3).
Instruction Courante	L'ordinateur est à l'étape : "Afficher le nombre actuel".

N.B :

- La **Pile** est utilisée pour se souvenir où revenir après avoir fait une tâche
- Le **Tas** est utilisé pour ranger les nouvelles choses créées pendant l'exécution

3 Chargement du Programme en Mémoire

Définition

Objectif : Placer le programme exécutable, stocké sur le disque, dans la mémoire physique pour l'exécuter. Le chargeur de l'OS doit également ajouter des segments pour gérer entre autres la pile et le tas.

Mémoire réelle (physique) : Correspond à la quantité de mémoire installée sur le système.

Elle peut être vue comme un tableau d'octets.

3.1 Mémoire Virtuelle

Définition

La **Mémoire Virtuelle** est une technique utilisée par le Système d'Exploitation (OS) qui permet à chaque processus d'avoir l'impression d'utiliser un espace d'adressage mémoire grand, contigu et privé.

Question : Comment l'OS gère plusieurs processus sans qu'ils s'écrasent ?

Réponse : Grâce à un système de pages virtuelles et de table des pages qui mappe l'espace logique vers la mémoire physique réelle.

3.2 Processus de Chargement

1. Diviser le programme en pages

Pour que l'OS puisse gérer la mémoire, il découpe le programme binaire (le code et les données) en petits morceaux de taille fixe. Ces morceaux sont appelés **Pages Virtuelles**.

2. Passage d'un adressage linéaire à un adressage en mémoire virtuelle

L'OS traduit ces adresses en **Adresses Virtuelles**, spécifiques à l'espace mémoire privé de ce processus.

3. Vérification de la disponibilité de cases en mémoire réelle

Vérification des espaces libres dans la RAM. Ces espaces s'appellent des **Cases Physiques** (Frames).

4. Chargement des pages utilisées dans les cases disponibles

L'OS ne charge pas tout le programme d'un coup. Il ne copie du disque vers la RAM que les pages dont le processus a immédiatement besoin pour commencer.

3.3 Exemple de Passage Adresse Linéaire → Adresse Virtuelle

Exemple

Contexte : Le code veut lire le compteur qui se trouve à 100 octets du début du segment DATA.

Hypothèse : Le programme est construit comme s'il commençait à l'adresse zéro.

Décision de l'OS : L'Adresse Virtuelle de Base (AVB) est fixée à 0x400000.

Calcul :

$$\text{Nouvelle Adresse Virtuelle} = \text{AVB} + \text{Décalage Linéaire}$$

$$100_{\text{déc}} = 64_{\text{hex}}$$

$$\text{Adresse} = 0x400000 + 0x000064 = 0x400064$$

4 Gestion des Processus par le Système

4.1 Commande ps

Note Importante

La commande **ps** liste les processus avec :

- Le propriétaire de chaque processus
- Le PID (Process ID)
- Le PPID (Parent Process ID)

Options utiles :

- **-e** : Tous les processus

- `axjf` : Arborescence des processus
 - `-u username` : Processus d'un utilisateur
- Plus d'options avec `man ps` !

5 États d'un Processus

Définition

Un processus peut prendre l'un de ces 3 états :

- **En cours (Running)** : Le programme utilise le processeur
- **Prêt (Ready)** : Exécutable, temporairement arrêté pour laisser un autre processus
- **Bloqué (Sleep)** : Ne peut pas s'exécuter tant qu'un événement externe ne se produit pas

Le passage de *En cours* à *Prêt* et inversement est géré par l'**ordonnanceur de processus**.

6 Ordonnancement des Processus

Définition

L'**ordonnanceur** est un composant clé du système d'exploitation chargé de décider quel processus doit s'exécuter à un moment donné. Son rôle est crucial pour assurer une gestion efficace du processeur.

Le système d'exploitation associe à chaque processus un bloc de contrôle appelé **Process Control Block (PCB)** qui contient :

- Un numéro unique (PID)
- L'état actuel du processus
- Pointeur vers la prochaine instruction
- Valeurs des registres
- Détails sur l'utilisation de la mémoire
- Détails sur les périphériques d'E/S utilisés
- Utilisation du CPU, priorité du processus

6.1 Les Niveaux d'Ordonnancement

Niveau	Description
Ordonnancement long terme (Job scheduling)	Décide des processus que le système peut mener en parallèle.
Ordonnancement à moyen terme (Swapper)	Assure la permutation des processus placés en mémoire.
Ordonnancement à court terme (Dispatcher)	Choisit à quel processus sera alloué le processeur et pour quel laps de temps.

6.2 Types d'Ordonnancement

Type	Description	Exemple
Non Préemptif	L'ordonnanceur attend que le processus actuel se termine ou se bloque.	Un serveur qui attend que chaque client termine sa commande.
Préemptif	L'ordonnanceur peut interrompre un processus pour en traiter d'autres.	Un serveur qui aide plusieurs clients à la fois.

7 Communication et Synchronisation des Processus

7.1 Problème : Condition de Course (Race Condition)

Attention

Scénario : Deux processus essaient tous deux de mettre à jour une ressource partagée (variable en mémoire ou fichier) sans coordination.

Problème : L'ordonnanceur peut interrompre un processus entre ses opérations, laissant la ressource dans un état incohérent.

Exemple

Exemple concret :

1. Processus A lit `Compteur` = 5
2. L'ordonnanceur le préempte
3. Processus B lit `Compteur` = 5, incrémente à 6, et écrit
4. L'ordonnanceur redonne la main à A
5. A ignore l'action de B, calcule $5 + 1 = 6$ et écrit
6. **Résultat : 6 au lieu de 7**

7.2 Techniques de Synchronisation

Note Importante

Il existe plusieurs techniques de communication et synchronisation entre les processus :

- | | |
|------------------------------|------------------------|
| — Condition de concurrence | — Les moniteurs |
| — La section critique | — L'échange de message |
| — Les sémaphores | — Les barrières |
| — Le sommeil et l'activation | |

7.3 Le Mutex (Exclusion Mutuelle)

Définition

Le **Mutex** (de l'anglais *Mutual Exclusion*) est l'outil le plus simple et le plus couramment utilisé pour résoudre les conditions de concurrence. Il agit comme une clé unique pour accéder à une ressource critique.

Concept : Garantir qu'un seul processus à la fois puisse exécuter la section critique (le code qui manipule la ressource partagée).

Mécanisme :

- Un processus doit prendre le **verrou (lock)** pour entrer dans la section critique
- S'il est déjà pris, le processus est bloqué (mis en attente)
- Une fois le travail terminé, le processus doit **libérer le verrou (unlock)**

7.4 Les Sémaphores

Définition

Le **Sémaphore** est un outil plus général. Il peut être utilisé pour l'exclusion mutuelle, mais aussi pour gérer l'accès à un nombre limité de ressources, ou pour signaler qu'un événement s'est produit.

Concept : Contrôler l'accès à une ressource ou coordonner l'exécution.

Mécanisme :

- **P() ou wait()** : Décrémente le compteur. Si le résultat est négatif, le processus est bloqué.
- **V() ou signal()** : Incrémente le compteur. Si le résultat devient positif ou nul, un processus bloqué est réveillé.

Types de Sémaphore :

- **Binaire** : Similaire à l'exclusion mutuelle
- **Compteur** : Utilisé pour limiter l'accès à N ressources identiques

8 Programme vs Processus

Concept	Description
Programme	Le code statique (les instructions) stocké sur le disque dur. <i>C'est la recette.</i>
Processus	Une instance d'un programme en cours d'exécution. <i>C'est la recette en cours de préparation dans la cuisine, avec ses ingrédients (données), son espace de travail (mémoire), et son chef (le CPU).</i>

8.1 Création de Processus

Fork	Crée un doublon exact du processus appelant (le parent), y compris son code, ses données, et son état.
	• Retourne 0 au processus enfant • Retourne le PID de l'enfant au processus parent
Exec	Est généralement appelé immédiatement après <code>fork()</code> par le processus enfant. Il remplace l'espace mémoire et le code du processus enfant par un nouveau programme.

Note Importante

Sous Windows : Vous pouvez utiliser la commande `start-process`

9 Exécution d'un Processus

9.1 Exécution en Série (Séquentielle)

Définition

Les processus s'exécutent l'un après l'autre. Le Processus B ne commence que lorsque le Processus A est terminé.

Synchronisation : La fonction `wait()` est utilisée par le parent pour bloquer son exécution jusqu'à ce que son enfant ait fini.

Exemple

Compiler d'abord un fichier source, puis exécuter le programme obtenu.

Sous Linux :

```
1 echo "Debut_de_la_Tache_1." ; sleep 5 ; echo "Fin_de_la_Tache_2."
```

9.2 Exécution en Parallèle (Concurrente)

Définition

Les processus s'exécutent simultanément (si plusieurs cœurs CPU sont disponibles) ou concurrentiellement (grâce à l'ordonnanceur).

Synchronisation : Les mécanismes Mutex/Sémaphore sont cruciaux pour gérer les accès aux données.

Exemple

Ouvrir un tableur, un navigateur web et un lecteur de musique en même temps.

Sous Linux :

```
1 commande1 & commande2 & commande3
```

Le symbole & permet de lancer la commande en arrière-plan, le terminal reste libre pour la commande suivante.

9.3 Exécution Pipe

Définition

Un type spécial d'exécution en série où la sortie d'un processus est directement connectée à l'entrée du processus suivant. C'est un tube de communication unidirectionnel (FIFO - First In, First Out).

Synchronisation : Utilisation de la communication inter-processus (IPC).

10 Algorithmes d'Ordonnancement

Note Importante

L'ordonnancement de Linux est fondé sur les **threads** et non sur les processus. Linux distingue trois classes de threads pour l'ordonnancement :

1. Les FIFO temps réel
2. Les tourniquets temps réel
3. Les threads en temps partagé

10.1 FCFS (First-Come First-Served)

Définition

Les jobs attendent dans une file. Le premier arrivé est admis immédiatement et s'exécute tant qu'il n'est pas bloqué ou terminé. Lorsqu'il se bloque, le processus suivant commence à s'exécuter et le processus bloqué va se mettre au bout de la file d'attente. C'est typiquement un algorithme **non préemptif**.

Avantage : Algorithme simple et équitable.

Inconvénient : Le processus qui utilise davantage de temps processeur est favorisé par rapport à ceux qui font beaucoup d'appels E/S.

10.2 Shortest Job First (SJF)

Définition

Le job le plus court d'abord sera élu, le processus dont on suppose que le traitement sera le plus court.

Si quatre jobs ont des durées d'exécution a , b , c et d :

- Le premier se termine à l'instant a
- Le second à l'instant $a + b$
- Et ainsi de suite

Le premier job exécuté contribue donc beaucoup plus que les autres à la durée moyenne. Il est donc normal d'exécuter le plus court en premier lieu.

Exemple

Analogie : C'est ce qui se passe quand à la caisse d'une grande surface les clients laissent passer devant quelqu'un qui n'a qu'un article.

10.3 Shortest Remaining Time (SRT)

Définition

C'est la version **préemptive** de l'algorithme précédent.

Exemple

Analogie : Cette fois le client qui n'a qu'un article vient carrément interrompre la caissière.

10.4 Round Robin

Définition

Chaque processus reçoit tour à tour un intervalle de temps appelé **quantum**. Au terme de ce quantum ou, si le processus s'achève ou se bloque avant cet instant, l'ordonnanceur attribue directement le processeur au processus suivant.

L'algorithme est simple et équitable. C'est généralement cet ordonnancement circulaire qui est utilisé dans les systèmes à temps partagé.

Valeur typique du quantum : 20 à 50 ms

Attention

- Des quanta **plus courts** provoqueraient trop de commutations de processus et la proportion du temps consacré à ces changements de contexte deviendrait trop importante.
- Des quanta **plus longs** pénaliseraient les temps de réponse aux processus interactifs.

10.4.1 Exemple : Round Robin

Exemple

Supposons que nous avons trois processus P1, P2, et P3 avec un **quantum de temps** : 2 unités :

- P1 a besoin de 5 unités de temps
- P2 a besoin de 3 unités de temps
- P3 a besoin de 4 unités de temps

Diagramme de Gantt :

P1	P2	P3	P1	P2	P3	P1
0-2	2-4	4-6	6-8	8-9	9-11	11-12

Temps d'attente pour chaque processus :

$$P1 : 12 - 5 = 7 \text{ unités}$$

$$P2 : 9 - 3 = 6 \text{ unités}$$

$$P3 : 11 - 4 = 7 \text{ unités}$$

Temps moyen d'attente :

$$\frac{7 + 6 + 7}{3} = 6.67 \text{ unités de temps}$$

10.5 L'Ordonnancement avec Priorité

Définition

Une valeur de priorité est assignée à chaque processus. Elle peut être fonction :

- D'un ordre de préséance entre utilisateurs
- Des caractéristiques des processus

Un processus temps réel sera par exemple prioritaire par rapport à une tâche de fond. La priorité peut varier dynamiquement.

Exemple

Priorité dynamique : Pour ne pas encombrer la mémoire avec des processus qui passent le plus clair de leur temps à attendre des entrées/sorties, on leur accorde une priorité d'autant plus grande qu'ils ne consomment qu'une petite fraction de leur quantum.

Si le quantum est fixé à 50 ms :

- Un processus qui n'utilise que 1 ms avant d'être bloqué aurait droit à une priorité de $\frac{50}{1} = 50$
- Un processus qui se bloque au bout de 25 ms a droit à une priorité de $\frac{50}{25} = 2$
- Les processus qui consomment tout le quantum ont une priorité de 1

10.5.1 Exemple : Ordonnancement avec Priorité

Exemple

Soit P1, P2, P3 :

- P1 a besoin de 5 unités de temps, avec une priorité de 2
- P2 a besoin de 3 unités de temps, avec une priorité de 1 (plus haute)
- P3 a besoin de 4 unités de temps, avec une priorité de 3

Ordre d'exécution : P2 → P1 → P3

Diagramme de Gantt :

P2	P1	P3
0-3	3-8	8-12

Temps d'attente :

$$P1 : 3 - 0 = 3 \text{ unités}$$

$$P2 : 0 - 0 = 0 \text{ unités}$$

$$P3 : 8 - 0 = 8 \text{ unités}$$

Temps moyen d'attente :

$$\frac{3 + 0 + 8}{3} = 3.67 \text{ unités}$$